

PharmaHub RAG Service Study Guide

A code-first walkthrough of the medical RAG pipeline in this repo.

Use this guide to explain the flow in an interview and to rebuild the service file by file without getting lost in the repo structure.

React UI

- > Express controller
- > safety check
- > Redis cache
- > ragClient.js
- > FastAPI /chat
- > retrieval.py
- > prompt_builder.py
- > generator.py
- > response with citations

The active medical assistant lives under my-react-app/backend/ and rag-service/. The root-level server.js is a legacy backend and is not the main RAG path.

1. What the RAG service does

RAG means retrieval-augmented generation. In this project, the assistant does not answer only from model memory. It first retrieves relevant medicine context from MongoDB, then builds a grounded prompt, then generates a safe answer, and finally returns citations and confidence metadata.

- Accept a medical question from the frontend.
- Stop emergency or high-risk messages early.
- Check Redis before doing expensive work.
- Retrieve the best context from the medical knowledge base.
- Build a prompt that forces grounded answers.
- Call Gemini when available, otherwise return a safe fallback.
- Return answer, citations, confidence, risk level, and urgent-care flags.

2. File map

This table shows the files you should know first when you explain or rebuild the RAG flow.

File	Job
my-react-app/backend/controllers/medicalAssistantController.js	Orchestrates validation, safety, cache, FastAPI call, and local fallback.
my-react-app/backend/services/ragClient.js	Sends HTTP requests to the Python RAG service and normalizes the response.
my-react-app/backend/routes/medicalAssistantRoutes.js	Registers the /chat and /health endpoints for the assistant.
my-react-app/backend/services/rag/safety.js	Checks emergency symptoms before the rest of the pipeline runs.
my-react-app/backend/services/rag/retrieval.js	Local keyword fallback if the Python service is unavailable.
rag-service/app.py	FastAPI entry point for chat and health checks.
rag-service/retrieval.py	Runs vector search and keyword search over MongoDB documents.
rag-service/prompt_builder.py	Turns retrieved chunks into a grounded prompt.
rag-service/generator.py	Calls Gemini or returns a safe fallback answer.
rag-service/ingest.py	Loads the PDF corpus, chunks it, embeds it, and writes it to MongoDB.
rag-service/pdf_loader.py	Extracts text from PDF pages.
rag-service/chunker.py	Splits long text into overlapping chunks.
rag-service/embed.py	Creates lightweight deterministic embeddings for the demo pipeline.
rag-service/config.py	Connects to MongoDB and exposes the medicine_vectors collection.

3. What to write first

If you were coding this from scratch, write the files in this order so the pieces fit together naturally.

- `config.py` - MongoDB connection and collection handle.
- `embed.py` - Embedding function used by ingestion and retrieval.
- `chunker.py` - Splits the raw source text.
- `pdf_loader.py` - Reads text from the PDF corpus.
- `ingest.py` - Stores chunks and embeddings in MongoDB.
- `retrieval.py` - Finds the best supporting chunks.
- `prompt_builder.py` - Builds the grounded prompt.
- `generator.py` - Uses Gemini or a safe fallback.
- `app.py` - Exposes the FastAPI chat contract.
- `ragClient.js` - Lets Node call the Python service.
- `medicalAssistantController.js` - Owns safety, caching, and fallback.

4. Request flow

This is the exact runtime story you can tell in an interview:

- The user types a question in the React medical assistant.
- Frontend API code sends the request to the Express backend.
- The controller validates that the message is not empty.
- The safety module checks urgent keywords like chest pain or seizure.
- If the message is urgent, the request stops immediately.
- If it is safe, the controller checks Redis for a cached answer.
- On a cache miss, the controller calls the Python RAG service.
- The Python service normalizes the text and checks emergencies again.
- Retrieval searches MongoDB using vector search plus keyword search.
- The prompt builder turns the retrieved snippets into a grounded prompt.
- The generator calls Gemini if configured, otherwise returns a safe fallback.
- The response returns citations, confidence, risk level, and status flags.
- The controller normalizes the payload and caches it for future requests.

```
User question
-> safety triage
-> Redis cache
-> askRagService()
-> FastAPI /chat
-> retrieve()
-> build_prompt()
-> generate_answer()
-> normalize response
-> cache and return
```

Interview point: the Node layer is the traffic cop. It does not generate the answer itself; it just guards, routes, caches, and falls back when needed.

5. Node side code

The Node backend is the orchestration layer. It is the place where validation, safety, Redis, and HTTP all come together.

ragClient.js

This file is a thin HTTP client. It sends the chat request to the FastAPI service and converts the response into one stable shape for the rest of the app.

```
const DEFAULT_BASE_URL = "http://127.0.0.1:8000";
const DEFAULT_TIMEOUT_MS = 15000;

function getRagServiceBaseUrl() {
  return normalizeBaseUrl(
    process.env.RAG_SERVICE_URL || process.env.RAG_SERVICE_BASE_URL
  );
}

export async function askRagService({ message, conversationId = null, locale = "en-US" } = {}) {
  const payload = { message, conversationId, locale };
  const response = await axios.post(
```

```

    `${getRagServiceBaseUrl()}/chat`,
    payload,
    { timeout: getTimeoutMs(), headers: { "Content-Type": "application/json" } }
  );

  const data = response.data || {};
  return {
    answer: typeof data.answer === "string" ? data.answer : "",
    citations: normalizeCitations(data.citations),
    confidence: typeof data.confidence === "number" ? data.confidence : null,
    riskLevel: data.riskLevel || "low",
    needsUrgentCare: Boolean(data.needsUrgentCare),
    requestId: data.requestId || Date.now().toString(),
    source: "fastapi",
  };
}

```

The key idea is that the client never assumes the backend shape is perfect. It normalizes citations, numbers, and optional fields so the frontend gets one clean contract.

medicalAssistantController.js

This is the most important Node file for the interview. It validates the message, runs safety checks, checks Redis, calls the Python service, and uses a local fallback if the Python layer is down.

```

export async function chat(req, res) {
  const { message, conversationId = null, locale = "en-US" } = req.body;
  if (!message || !message.trim()) {
    return res.status(400).json({ error: "Message is required" });
  }

  const normalized = message.trim();
  const safety = checkSafety(normalized);
  if (!safety.safe) {
    return res.json({
      answer: safety.answer,
      citations: [],
      riskLevel: safety.riskLevel,
      needsUrgentCare: safety.needsUrgentCare,
    });
  }

  const cacheKey = buildCacheKey(normalized);
  const cached = await redisClient.get(cacheKey);
  if (cached) {
    return res.json(JSON.parse(cached));
  }

  let retrieval;
  try {
    retrieval = await askRagService({ message: normalized, conversationId, locale });
  } catch (ragErr) {
    retrieval = {
      ...composeMedicalReply(normalized),
      source: "local-fallback",
      retrievalMode: "local-fallback",
      generationError: ragErr.message,
    };
  }

  return res.json(buildResponsePayload(retrieval, conversationId));
}

```

Interview point: the controller is where the system stays safe. It prevents empty input, blocks urgent cases, avoids repeated work with Redis, and still answers if the FastAPI service fails.

medicalAssistantRoutes.js

```
router.get("/health", async (_req, res) => {
  const ragHealth = await checkRagHealth();
  return res.json({
    backend: "ok",
    ragService: { status: "ok", details: ragHealth },
  });
});

router.post("/chat", chat);
```

6. Python side code

The Python service is where the RAG logic lives. It receives the question, finds the best evidence, builds the prompt, and generates the answer.

app.py

This is the FastAPI entry point. It defines the request and response models, checks for emergency keywords, calls retrieval, calls generation, and returns the final payload.

```
@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest):
    message = normalize_message(request.message)
    if not message:
        raise HTTPException(status_code=400, detail="Message is required")

    if has_emergency_keyword(message):
        return ChatResponse(
            answer=EMERGENCY_MESSAGE,
            citations=[],
            confidence=0.99,
            riskLevel="high",
            needsUrgentCare=True,
        )

    documents = retrieve(message, limit=3)
    prompt = build_prompt(message, documents)
    generation = generate_answer(prompt, question=message, documents=documents)

    confidence = estimate_confidence(
        documents, provider=generation.get("provider", "fallback"), urgent=False
    )
    risk_level = determine_risk_level(confidence, urgent=False)

    return ChatResponse(
        answer=generation.get("answer"),
        citations=build_citations(documents),
        confidence=confidence,
        riskLevel=risk_level,
        needsUrgentCare=False,
        generationProvider=generation.get("provider"),
    )
```

Important interview detail: confidence and riskLevel are derived from retrieval quality, not blindly copied from the model. That makes the API more predictable.

retrieval.py

This file performs hybrid retrieval. It tries vector search first, then keyword search, merges the results, scores them, and returns the best few chunks.

```
def retrieve(question, limit=3):
    query_embedding = generate_embedding(question)
    documents = []

    try:
        results = medicine_collection.aggregate([
            {
                "$vectorSearch": {
                    "index": "autoembed_index",
                    "path": "embedding",
                    "queryVector": query_embedding,
                    "numCandidates": 50,
                    "limit": limit,
                }
            }
        ])
```

```

    }
  ])
  documents = list(results)
except Exception:
  documents = []

keyword_documents = _fetch_keyword_documents(question, limit)
documents = _merge_documents(documents, keyword_documents)

scored_docs = []
for doc in documents:
  score, terms = _score_document(question, doc)
  scored_docs.append({
    "name": doc.get("name") or doc.get("source") or "medical-chunk",
    "source": doc.get("source"),
    "text": doc.get("text", ""),
    "score": score,
    "snippet": _build_snippet(doc, terms),
  })

scored_docs.sort(key=lambda item: item["score"], reverse=True)
return scored_docs[:limit]

```

Interview point: this is hybrid retrieval, not pure vector search. That matters because drug names and short product names often match better with keyword search.

prompt_builder.py

```

def build_prompt(question, documents):
  if not documents:
    return (
      "You are PharmaHub Medical Assistant.\n"
      "If there is no relevant local context, give a short general answer "
      "for low-risk topics.\n"
      "Do not diagnose or prescribe.\n\n"
      f"User question:\n{question}\n"
    )

  context_blocks = []
  for i, doc in enumerate(documents, start=1):
    context_blocks.append(
      f"Source {i}: {doc.get('name')}\n"
      f"Relevant snippet: {doc.get('snippet')}"
    )

  return (
    "You are PharmaHub Medical Assistant.\n"
    "Use the retrieved context as your source of truth.\n"
    "Do not invent facts.\n\n"
    f"User question:\n{question}\n\n"
    "Retrieved context:\n"
    f"{chr(10).join(context_blocks)}\n"
  )

```

The prompt tells the model how to behave. It uses retrieved context as the source of truth, asks for short safe answers, and refuses diagnosis or dosage changes.

generator.py

```

def generate_answer(prompt, question, documents):
  api_key = os.getenv("GEMINI_API_KEY", "").strip()
  model = os.getenv("GEMINI_MODEL", DEFAULT_MODEL).strip() or DEFAULT_MODEL

  if not api_key:
    return {
      "answer": _build_fallback_answer(question, documents),
      "provider": "fallback",
    }

```



```

        "model": model,
    }

    try:
        answer = _call_gemini(prompt, api_key, model)
        return {
            "answer": answer,
            "provider": "gemini",
            "model": model,
        }
    except Exception as exc:
        return {
            "answer": _build_fallback_answer(question, documents),
            "provider": "fallback",
            "model": model,
            "error": str(exc),
        }

```

The generator never hard fails for the user. If Gemini is missing or rate limited, it returns a safe fallback answer instead of breaking the chat flow.

7. Ingestion and PDF flow

The ingestion path is separate from the request path. It prepares the knowledge base that retrieval uses later. In this repo, the sample corpus is the PDF at `rag-service/data/product_catalog.pdf`.

pdf_loader.py

```
def extract_text(pdf_path):
    pdf_path = Path(pdf_path)
    if not pdf_path.exists():
        raise FileNotFoundError(f"PDF not found: {pdf_path}")

    reader = PdfReader(str(pdf_path))
    pages = []
    for page in reader.pages:
        text = page.extract_text() or ""
        if text.strip():
            pages.append(text.strip())
    return "\n\n".join(pages)
```

chunker.py

```
def chunk_text(text, chunk_size=800, overlap=120):
    cleaned = " ".join(text.split())
    words = cleaned.split()
    chunks = []
    start = 0

    while start < len(words):
        end = start + chunk_size
        chunk_words = words[start:end]
        if not chunk_words:
            break
        chunks.append(" ".join(chunk_words))
        start += chunk_size - overlap

    return chunks
```

embed.py

```
def generate_embedding(text):
    tokens = _tokenize(text)
    vector = [0.0] * EMBEDDING_DIMENSIONS

    for index, token in enumerate(tokens):
        _accumulate(vector, token, 1.0)
        if index + 1 < len(tokens):
            _accumulate(vector, f"{token}_{tokens[index + 1]}", 0.75)

    norm = math.sqrt(sum(value * value for value in vector)) or 1.0
    return [round(value / norm, 6) for value in vector]
```

Important note: this project uses a lightweight deterministic embedding for the demo. It is useful for local testing, but in production you would usually swap in a real embedding model.

ingest.py

```
def ingest_pdf(pdf_path=PDF_PATH):
    text = extract_text(pdf_path)
    chunks = chunk_text(text)

    if not chunks:
        raise ValueError("No text extracted from PDF")

    medicine_collection.delete_many({"source": pdf_path.name})
```

```

for chunk_id, chunk in enumerate(chunks, start=1):
    embedding = generate_embedding(chunk)
    medicine_collection.insert_one({
        "source": pdf_path.name,
        "chunk_id": chunk_id,
        "text": chunk,
        "embedding": embedding,
    })

```

The ingestion job reads the PDF, splits it into overlapping chunks, embeds each chunk, and writes the records to MongoDB so the retrieval step can search them later.

The optional PDF generator in `rag-service/generate_product_pdf.py` can rebuild the sample catalog from `my-react-app/backend/seedProducts.js` if you want fresh demo data.

8. Interview talk track

Here is a short explanation you can say out loud in an interview:

I built the medical assistant as a RAG pipeline. The Node controller handles validation, safety checks, and caching. It then calls a FastAPI service. The FastAPI service retrieves relevant medical chunks from MongoDB, builds a grounded prompt, and generates the answer with Gemini or a safe fallback. The response always includes citations, confidence, and a risk level so the UI can show the user how trustworthy the answer is.

- Safety first: emergency symptoms are blocked before normal retrieval.
- Hybrid retrieval: vector search plus keyword search.
- Grounded prompting: the model sees retrieved snippets, not raw memory.
- Safe generation: Gemini is optional; fallback text always exists.
- Traceability: citations and chunk IDs let you explain where the answer came from.

9. Environment variables

- MONGO_URI - MongoDB connection string.
- REDIS_URL - Redis cache connection string.
- RAG_SERVICE_URL - URL of the Python service.
- RAG_SERVICE_TIMEOUT_MS - Timeout for RAG HTTP calls.
- RAG_SERVICE_PORT - Port used by FastAPI.
- GEMINI_API_KEY - Enables Gemini generation.
- GEMINI_MODEL - Gemini model name.

If you want one simple debugging checklist, use this order: check Mongo, check Redis, check the FastAPI health endpoint, then test one known question, then confirm the response contains citations and a confidence score.

10. Common debugging points

- If the Python service is down, the Node controller falls back to the local medicine knowledge base.
- If the question is empty, the controller returns HTTP 400.
- If the message contains urgent symptoms, the system returns an emergency response immediately.
- If vector search fails, retrieval still tries keyword search.

- If Gemini is missing or rate limited, the generator returns safe fallback text.

That is the whole RAG service in one document: safety, retrieval, prompt building, generation, and response normalization.